Kids Motion Game

Requirements, Hardware and Module Plan

A camera-tracked, full-body interactive game platform for the family: kids jump, duck and lean in the room, and the game responds on the TV with score, health, power-ups and sound. Built as reusable modules so every new game reuses most of what already exists.

Document status	Draft for review (discussion stage, no code yet)
Purpose	Agree requirements, hardware, module list and build sequence before writing use cases
Next document	Full use-case catalogue: every success, failure and edge case per module
Tech baseline	Python and VS Code on a ThinkPad (i5 12th gen, 8 GB RAM); OpenCV, MediaPipe, pygame

How to read this document

- Part 1: what we want to achieve (requirements).
- Part 2: hardware we own, the role of each device, and what may be bought later.
- Part 3: how the whole system fits together (architecture and data flow).
- Part 4: every module in build order, with purpose, responsibilities, inputs, outputs, dependencies and why it sits where it does.
- Part 5: milestones and the review checklist: the decisions you should confirm or change before we write use cases.

Part 1: Requirements

1.1 Vision

Turn a normal room into a game console. A child stands on marked floor tiles in front of the TV. A camera watches the child's body. When the child jumps, ducks or leans, the on-screen character does the same. The TV shows the road, obstacles, power-ups, score and a health bar, and the TV speakers play sound effects. The same platform later runs many different games (runner, floor is lava, balloon pop, freeze dance) and finally moves the play surface from the TV to a projected floor.

1.2 Goals

- Play a runner-style game with the whole body: jump over obstacles, duck under others, lean to change lanes.
- Reward success with points, power-ups and speed increases; punish contact with health loss and sound.
- Recognise each child, remember their body measurements and avatar, and use the same profile in every game.
- Make adding a new game cheap: a new game should mostly be new rules and art, not new plumbing.
- Be safe, forgiving and fun for children first, and a good engineering project for the parent second.
- Grow in stages: TV first, then floor projection, without rewriting the core.

1.3 Functional requirements

A. Gameplay

- FR-A1: A scrolling world with a character, obstacles and collectibles.
- FR-A2: Actions: jump, duck, lean left, lean right (and later: step, hands up, pose match).
- FR-A3: Points for each obstacle cleared; bonus for streaks.
- FR-A4: Health bar that drops by a small amount on obstacle contact; game over at zero.
- FR-A5: Power-ups on the road (for example shield, slow-down, double points, health refill) with clear on-screen feedback and a time limit where relevant.
- FR-A6: Speed ramps gradually with progress, with a cap so the game stays playable.
- FR-A7: States: menu, profile select, calibration, countdown, playing, paused, game over, results.
- FR-A8: Local high scores per profile and per game.

B. Sound

- FR-B1: Distinct sounds for jump, landing, movement or running loop, obstacle hit, power-up pickup, health low, game over, menu clicks and a countdown.
- FR-B2: Sound plays through the TV speakers via HDMI (no extra speakers owned).
- FR-B3: Sound latency must be low enough that a jump sound feels simultaneous with the jump.

C. Body tracking and action detection

- FR-C1: Detect a person from a camera and extract body landmarks in real time.
- FR-C2: Lock on to one player; ignore other people walking through the frame.
- FR-C3: Detect JUMP, DUCK, LEAN_LEFT, LEAN_RIGHT reliably with smoothing to suppress jitter and false triggers.
- FR-C4: Use per-child thresholds (height, jump baseline) so detection is fair for a small child and an adult.
- FR-C5: Know the play zone (tile boundary) and react when the player steps outside it (pause or warn).

- FR-C6: Work with one camera; improve with two or three (front, side, laptop).

D. Profiles and identity

- FR-D1: Create a profile with a name, avatar choices and body measurements captured by the camera.
- FR-D2: Load a saved profile at the start of any game so the character looks and scales the same.
- FR-D3: Re-measure on demand, since children grow.
- FR-D4: One universal profile file per person, shared across all games, including per-game high scores.
- FR-D5: Face detection is allowed as a helper (for example to confirm someone is present and facing the camera); face recognition (automatically knowing who it is) is optional and deferred. Default selection is by choosing a name on the TV.
- FR-D6: Store numbers and settings, not face photos.

E. Setup, testing and diagnostics

- FR-E1: Every game must be playable with the keyboard, so games can be built and tested without cameras.
- FR-E2: Live debug view showing camera frame, skeleton overlay, detected events and frame rate.
- FR-E3: Record and replay tracking sessions to reproduce and fix detection problems.
- FR-E4: One configuration file for thresholds, camera assignments, display and audio settings.
- FR-E5: Logs that explain what happened and why when something fails.

1.4 Non-functional requirements

Area	Requirement
Latency	Camera-to-reaction under about 150 ms for wired cameras (target); games are forgiving so up to about 250 ms is still playable.
Frame rate	Game runs at 60 FPS on the TV; tracking runs at 20 to 30 FPS independently and must never slow the game.
Resources	Must run on 8 GB RAM, integrated graphics. Game renders at 1080p even though the TV is 4K.
Reliability	Camera disconnects, tracking loss and crashes must degrade gracefully (pause, message, recover), never freeze the screen.
Safety	Short sessions, a clear zone, soft landings, no obstacles near the play area, and a pause on leaving the zone.
Privacy	No images stored by default. Profiles hold numbers and names only. Debug recordings are opt-in and kept local.
Reusability	New game requires changes only in the Mechanics Library and a new game module.
Maintainability	Modules talk only through the Event Bus and shared data structures; no module reaches into another's internals.
Usability	A child can start a game alone: walk up, pick own name, stand on the centre tile, see a countdown, play.

1.5 Constraints and assumptions

- Single laptop does everything: capture, tracking, game and display.
- TV is 4K 43 inch, on a stand against a wall, connected by HDMI through the USB-C hub, used as an extended display.
- Cameras are consumer phones via webcam apps (Iriun or DroidCam) plus the laptop webcam. Phone-as-webcam quality, latency and multi-phone support must be tested early.
- Indoor lighting is reasonably even; no strong window light behind the player.

- Players are children, so unpredictable movement, differing sizes and short attention spans are normal.
- No code is written until requirements, modules and use cases are agreed.

1.6 Out of scope for now

- Floor projection (planned later as a new output stage).
- Apple Watch or phone motion sensors as input.
- Online play, accounts, leaderboards across devices.
- Face recognition login.
- Multi-player simultaneous tracking (designed for later; one active player first).

1.7 Success criteria

Milestone	How we know it works
M1 Keyboard game	Family can play the runner with the keyboard: score, health, power-ups, speed ramp, sounds all working.
M2 Body control	A child jumps and the character jumps, with fewer than about 1 false trigger per minute and most real jumps detected.
M3 Profiles	A child picks their name, is measured once, and the same character and thresholds come back next day.
M4 Better cameras	Front phone camera at 9 to 10 ft gives noticeably steadier tracking than the laptop webcam.
M5 Second and third game	A new game is built mostly from existing modules in a fraction of the first game's effort.
M6 Floor ready	Output module can target a projector and calibrate to the floor without changing the games.

Part 2: Hardware

2.1 What you already own and its role

Device	Specification	Role in the system
Laptop	ThinkPad, Intel i5 12th gen, 8 GB RAM, Python and VS Code installed	The brain: runs camera capture, pose tracking, game logic, audio and display. Webcam doubles as a camera.
TV	Xiaomi TV A Pro 2025, 43 inch, 4K	Main game screen and sound output. Use Game Mode. Game renders at 1080p.
USB-C hub	Selore S Global: HDMI (4K), USB-A, USB-C, LAN, serial, 3.5 mm jack	Connects TV (HDMI) and cameras (USB). Shares bandwidth, so test under load.
Phone 1	Xiaomi Poco X6 Pro (proposed)	Front camera on a tripod by the TV, portrait: main player tracking.
Phone 2	Xiaomi Poco X3 NFC (proposed)	Side camera mid-zone: jump height and landing confirmation (optional at first).
Laptop webcam	Built into the ThinkPad	Development, profile setup at close range, menu navigation, backup angle.
Cables	HDMI, many USB-C cables	HDMI laptop to TV via hub; USB-C for phones. Keep runs under about 3 m.
Floor tiles	2 ft x 2 ft tiles	Zone marking and a real-world scale reference for measurements.
Spare devices	Galaxy Tab S9, iPhone 16 Pro, Apple Watch Series 10	Not used initially. Possible later as extra cameras or sensors.

2.2 Room layout (proposed)

- The TV stand (about 2 ft deep) is against one wall. The opposite wall is about 24 ft away, leaving about 22 ft of usable depth.
- **Play zone:** 3 tiles wide by 2 tiles deep (6 ft x 4 ft), marked with coloured tape, with a distinct centre start tile. Jump games use the centre tile; lane games use all three columns.
- **Distance:** centre of the zone about 9 to 10 ft from the TV. That keeps the 43 inch screen readable and gives the front camera room to see a whole jumping child.
- **Clearance:** about 3 ft free on each side of the zone and roughly 12 ft behind it.
- **Front camera:** on a small tripod directly below or beside the TV, portrait orientation.
- **Side camera (later):** about 8 to 10 ft to the side of the zone, mid-depth, at roughly waist to chest height.
- **Laptop:** beside the TV stand, close to the front camera so USB cables stay short.
- **Lighting:** even light on the player from the front or above; no window or bright lamp behind the player.
- **Floor:** non-slippery; socks with grip or barefoot on a safe surface; nothing breakable nearby.

2.3 Possible purchases (not needed to start)

Item	Why	Rough cost (AED, verify before buying)
Phone tripod or stand (x2)	Stable, aimable camera mounts. Needed once phones replace the laptop webcam.	30 to 120 each
Longer or better USB cable	For the side camera if it sits beyond about 3 m	20 to 80
Small speaker (wired)	Only if TV audio proves laggy or weak	80 to 250

Item	Why	Rough cost (AED, verify before buying)
Floor projector	Stage 2: projected road and lava on the floor	1,000 to 2,000 for a sensible model
Ceiling or shelf mount	Hold the projector pointing down	50 to 150
Coloured floor tape	Zone marking	10 to 30

2.4 Software baseline

- **Python** (check which version is installed; some tracking libraries support only certain versions, so confirm compatibility before starting).
- **OpenCV** for camera access and image handling.
- **MediaPipe** for body pose (and face landmarks if we use them).
- **pygame** for the game window, input, sprites and audio.
- **Iriun or DroidCam** for phone-as-webcam. Test with one phone first, then two.
- A simple local file format (JSON) for profiles, config and recordings.

2.5 Known hardware risks to test first

- Whether two phones can run as webcams at once on this laptop and hub.
- USB bandwidth and heat on the hub with two cameras and HDMI together.
- Wireless phone camera latency (100 to 300 ms); prefer USB.
- Whether a portrait phone at 9 to 10 ft frames a whole child plus jump headroom.
- Whether the laptop's integrated graphics keeps 60 FPS while tracking runs.
- Whether the TV's Game Mode removes noticeable audio and video delay.

Part 3: Architecture

3.1 Data flow

Camera Manager -> Pose Tracker -> Player Lock -> Zone and Calibration -> Action Detector -> **Event Bus** -> Game Engine and Game -> Display and UI, Audio Manager.
The keyboard adapter publishes the same events into the same Event Bus, so the game cannot tell whether a jump came from a key or a child.

3.2 Design principles

- **Events, not calls.** Modules publish and subscribe to events on the Event Bus instead of calling each other directly.
- **Separation of input and game.** The game never knows about cameras. It sees only events such as JUMP.
- **Independent loops.** Capture and tracking run at their own pace; the game loop always renders smoothly.
- **Replaceable parts.** A camera source, a tracker or a display can be swapped (phone for laptop webcam, TV for projector) without touching the rest.
- **Config over constants.** Thresholds and settings live in one file so tuning does not mean editing code.
- **Fail soft.** Every module defines what it does when its input is missing or bad.
- **Observable.** Everything important is logged and visible in the debug view.

3.3 Draft event catalogue

Group	Events (names may change when we write use cases)
Player actions	JUMP, LAND, DUCK, STAND, LEAN_LEFT, LEAN_RIGHT, LEAN_CENTER, HANDS_UP, STEP_LEFT, STEP_RIGHT, POSE_MATCHED
Player state	PLAYER_DETECTED, PLAYER_LOST, PLAYER_LOCKED, PLAYER_IN_ZONE, PLAYER_OUT_OF_ZONE, MULTIPLE_PEOPLE
Camera	CAMERA_CONNECTED, CAMERA_LOST, CAMERA_LOW_FPS, CAMERA_SWITCHED
Game flow	GAME_START, GAME_PAUSE, GAME_RESUME, GAME_OVER, STATE_CHANGED, COUNTDOWN_TICK
Gameplay	OBSTACLE_CLEARED, OBSTACLE_HIT, POWERUP_COLLECTED, POWERUP_EXPIRED, HEALTH_CHANGED, SCORE_CHANGED, SPEED_CHANGED, STREAK_CHANGED
Profile	PROFILE_SELECTED, PROFILE_CREATED, PROFILE_UPDATED, MEASUREMENT_STARTED, MEASUREMENT_DONE, MEASUREMENT_FAILED
System	CONFIG_LOADED, ERROR, WARNING, DEBUG_TOGGLED

3.4 Core shared data (conceptual)

- **Landmarks:** a timestamped list of body points per frame, per camera.
- **Player state:** position in zone, standing baseline, current posture.
- **Profile:** name, avatar options, measurements, jump baseline, thresholds, high scores per game.
- **Game state:** current state machine state, score, health, speed, active power-ups.
- **Config:** all tunable numbers, camera mapping, display and audio settings.

Part 4: Modules in build order

The order below is deliberate. Each phase ends with something that runs, so you get a working result early and every later step can be tested against it. Each module lists its purpose, responsibilities, inputs, outputs, dependencies, the reason for its position, the main risks and a plain 'done when' check.

1. Config and Logging

Phase A: Foundation (no camera, no game)

Purpose	One place for every tunable setting and one place where every module reports what it is doing.
Inputs	Settings file, command-line overrides.
Outputs	Settings available to all modules; log file and console output.
Depends on	Nothing.

Responsibilities

- Load a single settings file at startup with sensible defaults when values are missing.
- Validate values (ranges, types) and report clearly when something is wrong.
- Provide named settings: camera assignments, thresholds, display index, audio volume, debug flags.
- Provide logging with levels (info, warning, error) to screen and to a file.
- Rotate or limit log files so they never fill the disk.

Why it is at this position

Every later module needs settings and logs. Building it first means no module ever hard-codes a number or hides a failure, and tuning later is editing a file instead of code.

Main risks

- Settings sprawl if not organised by module from day one.
- Silent defaults hiding a typo in the file.

Done when

Any module can read a setting by name, a bad setting gives a clear error, and logs show up in a file with timestamps.

2. Event Bus

Phase A: Foundation (no camera, no game)

Purpose	The shared messenger. It defines every event, who may publish it and who may listen.
Inputs	Events from any module.
Outputs	Events delivered to subscribers; optional event log.
Depends on	Config and Logging.

Responsibilities

- Publish and subscribe with named events and a small data payload (timestamp, source, values).
- Define and document the event catalogue as the contract between modules.
- Deliver events without one slow listener blocking the others.
- Support a debug tap that shows or records every event.
- Guard against unknown event names and bad payloads.

Why it is at this position

It is the spine of the architecture. Every other module is written against it, so it must exist and be stable before anything else is built.

Main risks

- Overly chatty events hurting performance.
- Ordering problems when events arrive from different threads.

Done when

A test publisher and a test subscriber exchange events reliably, including when one subscriber is slow or crashes.

3. Input Adapter (Keyboard)

Phase A: Foundation (no camera, no game)

Purpose	A stand-in player. It turns key presses into the same events the camera will later produce.
Inputs	Keyboard state.
Outputs	Player action events on the Event Bus.
Depends on	Event Bus, Config.

Responsibilities

- Map keys to events (space to jump, down to duck, left and right to lean).
- Support press and release, so duck can be held.
- Make the key map configurable.
- Stay available permanently as a testing and fallback input.

Why it is at this position

It lets the whole game be built and tested before any computer vision exists. It also remains a safety net: if tracking misbehaves you can still play.

Main risks

- Keyboard feel (instant, perfect timing) hides timing problems that cameras will later expose; later phases must test with real latency.

Done when

Pressing keys produces the exact events the game will later receive from the Action Detector.

4. Display and UI

Phase B: Playable game with keyboard

Purpose	Everything the family sees on the TV: window, full-screen on the second display, game HUD and menus.
Inputs	Game state and events.
Outputs	Pixels on the TV.
Depends on	Config, Event Bus.

Responsibilities

- Open a full-screen window on the TV (extended display) at 1080p.
- Draw the HUD: score, health bar, speed, active power-ups, streak.

- Draw menus: main menu, profile select, pause, game over, results.
- Draw feedback: hit flash, power-up pop, low-health warning.
- Provide a drawing layer abstraction so a projector floor view can be added later as another output.

Why it is at this position

Nothing can be judged without seeing it. Building the display early lets us design the HUD and menus while there is no tracking noise to confuse the picture.

Main risks

- Wrong display chosen or resolution mismatch on the 4K TV.
- Tearing or low frame rate on integrated graphics.

Done when

A full-screen window appears on the TV, runs at a steady 60 FPS and a test HUD updates from events.

5. Audio Manager

Phase B: Playable game with keyboard

Purpose	Plays every sound effect and music track in response to events.
Inputs	Events; sound files.
Outputs	Audio output on the TV.
Depends on	Event Bus, Config.

Responsibilities

- Map events to sounds (jump, land, hit, pickup, low health, game over, click, countdown).
- Handle overlapping sounds, volume levels and mute.
- Loop a running or background sound and stop it cleanly.
- Preload sounds so there is no delay on first play.
- Use the TV speakers through HDMI.

Why it is at this position

Sound is half of the game feel and is cheap to add once events exist. Testing it before tracking separates audio delay from tracking delay.

Main risks

- TV audio latency; verify with Game Mode.
- Sounds clipping or cutting each other off.

Done when

Every mapped event plays its sound instantly, overlapping sounds mix cleanly and volume is adjustable.

6. Game Engine Core

Phase B: Playable game with keyboard

Purpose	The reusable heart of every game: loop, state machine, timing and difficulty ramp.
Inputs	Events, config.
Outputs	State changes and game ticks.
Depends on	Event Bus, Display, Audio, Config.

Responsibilities

- Run a fixed-timestep game loop independent of camera speed.
- Manage game states (menu, profile select, calibration, countdown, playing, paused, game over, results) and legal transitions.
- Provide timers, countdowns and a difficulty curve (speed over time or distance).
- Handle pause and resume, including auto-pause when the player leaves the zone.
- Expose a simple game interface that each game implements (setup, update, draw, on_event, teardown).

Why it is at this position

It is the part reused by every future game, so it should be built once, cleanly, before the first game's specific rules.

Main risks

- Letting game-specific logic leak into the engine, which kills reuse.

Done when

A blank game can move through all states, pause and resume, and a difficulty value rises smoothly over time.

7. Game Mechanics Library

Phase B: Playable game with keyboard

Purpose	Reusable gameplay building blocks that any game can combine.
Inputs	Game state, player action events.
Outputs	Gameplay events and drawable objects.
Depends on	Game Engine Core, Event Bus.

Responsibilities

- Health and damage with invulnerability frames after a hit.
- Score, streaks and multipliers.
- Obstacle spawning, movement, patterns and removal.
- Collision checks with forgiving hitboxes.
- Power-up types, pickup, duration and expiry.
- Scrolling background and speed scaling.
- Event emission for every outcome (cleared, hit, collected).

Why it is at this position

This is where the long-term payoff sits. Games 2 and 3 should be mostly different combinations of these blocks, so they are built before the first game consumes them.

Main risks

- Hitboxes too strict for kids; make forgiveness a setting.
- Spawn patterns that are impossible to clear.

Done when

Blocks work together in a throwaway test scene: spawn, clear, hit, collect, ramp speed.

8. Game 1: Runner

Phase B: Playable game with keyboard

Purpose	The first complete game: a scrolling road, jump and duck obstacles, lane changes, power-ups, health, score and speed ramp.
Inputs	Player action events from the keyboard adapter.
Outputs	A complete playable game.
Depends on	Modules 1 to 7.

Responsibilities

- Define the road, character, obstacle set and power-ups.
- Tune difficulty for children.
- Wire events to sounds and HUD.
- Provide a results screen with score and personal best.

Why it is at this position

It proves the architecture and the fun. If the game is not fun with a keyboard, cameras will not fix it, so we find out before the hard part.

Main risks

- Game feels too easy or too hard; needs playtesting with the kids.

Done when

MILESTONE M1: the family can play start to finish with keys.

9. Camera Manager

Phase C: Eyes (tracking)

Purpose	Finds, opens and supervises every camera and hands frames to the rest of the system.
Inputs	Camera devices, config.
Outputs	Timestamped frames; camera status events.
Depends on	Config, Event Bus.

Responsibilities

- List available cameras and map them to roles (front, side, laptop) from config.
- Open cameras at the configured resolution and frame rate.
- Run capture in its own thread so a slow camera never freezes the game.
- Detect disconnects and slow frames; publish camera events; attempt reconnect.
- Provide the latest frame with a timestamp on request.
- Start with the laptop webcam only; add phones later without changing callers.

Why it is at this position

Everything in the vision chain starts with a reliable frame. Starting with the laptop webcam removes phone-app uncertainty from the first tracking work.

Main risks

- Camera index changes after reboot or replug.
- Two cameras on one hub exceeding bandwidth.
- Webcam app quirks with phones.

Done when

The laptop camera streams steadily, and unplugging it produces a clear event and a clean recovery.

10. Debug and Replay Tools

Phase C: Eyes (tracking)

Purpose	Lets us see what the tracker sees and replay problems instead of recreating them.
Inputs	Frames, landmarks, events.
Outputs	Overlay window; recording files.
Depends on	Camera Manager, Event Bus.

Responsibilities

- Show the camera view with skeleton overlay, zone outline and detected events.
- Show frame rate, latency and key signals (hip height, shoulder tilt) as live graphs or numbers.
- Record landmark data (not video by default) from a session and play it back through the Action Detector.
- Toggle on and off with a key.
- Allow optional video recording that is local only and off by default.

Why it is at this position

Tracking problems are impossible to fix blind. Having this before the tracker means every later module is built with visibility.

Main risks

- Overlay slowing the system; make it lightweight and optional.

Done when

A recorded jump session can be replayed and produces the same events each time.

11. Pose Tracker

Phase C: Eyes (tracking)

Purpose	Turns a camera frame into body landmarks.
Inputs	Frames.
Outputs	Landmarks with confidence per frame.
Depends on	Camera Manager.

Responsibilities

- Run pose detection on frames at a steady 20 to 30 FPS.

- Output normalised landmarks with confidence values.
- Handle no person, partial body and low confidence.
- Choose a model size that suits the laptop.
- Prepare for multiple cameras, each with its own landmark stream.

Why it is at this position

The Action Detector, Zone and Player Lock all need landmarks. Debug tools come first so we can see this module's output the moment it exists.

Main risks

- CPU load on 8 GB RAM; reduce resolution if needed.
- Reduced accuracy on small bodies and in loose clothing.
- Library and Python version compatibility.

Done when

A person in front of the laptop produces a stable skeleton at the target frame rate.

12. Zone and Calibration

Phase C: Eyes (tracking)

Purpose	Knows where the play zone is and how image pixels relate to real distances.
Inputs	Landmarks, frames, config.
Outputs	Zone geometry, scale factors, in-zone state events.
Depends on	Pose Tracker, Camera Manager.

Responsibilities

- Define the tile zone from the camera view using marked reference points or tile corners.
- Convert pixel positions to approximate floor positions and real size using the 2 ft tiles as scale.
- Decide whether the player is inside, at the edge, or outside the zone.
- Save calibration per camera and per room setup.
- Offer a simple guided calibration routine.

Why it is at this position

Jump and lean thresholds only mean something once position and scale are known, so this must come before action detection.

Main risks

- Camera moved after calibration; detect and ask for recalibration.
- Perspective error from a low camera angle.

Done when

The system correctly reports inside, edge and outside as a child steps around the tiles.

13. Player Lock

Phase C: Eyes (tracking)

Purpose	Chooses one person as the player and keeps tracking that person.
Inputs	Landmarks (possibly several people), zone.
Outputs	Single-player landmark stream; lock events.
Depends on	Pose Tracker, Zone and Calibration.

Responsibilities

- Pick the player: the person standing in the zone, nearest the centre tile, for a set time.
- Ignore other people and objects.
- Keep identity through brief occlusion and crossings.
- Release the lock when the player leaves, and allow a new lock.
- Report multiple-people situations.

Why it is at this position

With kids and parents moving in a room, a wrong person controlling the game is the most visible failure. It sits right before the detector so the detector sees only the player.

Main risks

- Sibling stands next to the player and swaps the lock.
- Parent walks through the zone.

Done when

A second person passing the camera does not disturb the game.

14. Action Detector

Phase C: Eyes (tracking)

Purpose	Converts the player's landmarks into game events.
Inputs	Locked player landmarks, zone, profile thresholds.
Outputs	Player action events.
Depends on	Pose Tracker, Zone and Calibration, Player Lock, Event Bus.

Responsibilities

- Detect JUMP and LAND from vertical motion relative to a standing baseline.
- Detect DUCK, LEAN_LEFT, LEAN_RIGHT from body geometry.
- Smooth signals and apply debounce so jitter does not cause false events.
- Use per-profile thresholds when available, defaults otherwise.
- Avoid double-firing and handle rapid sequences.
- Publish the same events as the keyboard adapter.

Why it is at this position

It is last in the vision chain because it relies on everything before it. It is the module whose results the kids actually feel.

Main risks

- Latency making jumps feel late.
- False jumps from camera noise.
- Different children needing different thresholds.

Done when

MILESTONE M2: the kid jumps, the runner jumps, with very few false triggers.

15. Profile Manager

Phase D: Identity (profiles)

Purpose	Creates, stores, selects and updates the universal player profiles.
Inputs	User choices, measurement results.
Outputs	Profile data; profile events.
Depends on	Config, Event Bus, Display and UI.

Responsibilities

- Create, rename, duplicate-check and delete profiles.
- Store name, avatar choices, measurements, jump baseline, thresholds and per-game high scores.
- Load the selected profile and broadcast it so the game and detector use it.
- Keep the file format versioned so old profiles survive changes.
- Protect against corrupt files with automatic backup.

Why it is at this position

Profiles are useful only once there are measurements to store and thresholds to improve, so they follow a working detector.

Main risks

- Profile file corruption.
- Name collisions between siblings.

Done when

A profile survives restart and the same character and thresholds are applied in the game.

16. Measurement Capture

Phase D: Identity (profiles)

Purpose	The guided routine that measures a child and fills a new profile.
Inputs	Landmarks, zone, scale, UI.
Outputs	Measurements stored in the profile.
Depends on	Pose Tracker, Zone and Calibration, Profile Manager, Display and UI.

Responsibilities

- Guide the child to the start tile and a standard pose (stand tall, arms out).
- Measure height, shoulder width, limb proportions and standing baseline using the tile scale.
- Run a short test jump to set a personal jump baseline.
- Optionally use face landmarks as a helper for presence and facing direction.

- Validate quality and retry with plain instructions on failure.
- Let the parent type in true height to improve scale.

Why it is at this position

It needs a stable tracker and a place to store results, so it comes right after the Profile Manager.

Main risks

- Child will not stand still.
- Camera too close or too far gives bad numbers.

Done when

MILESTONE M3: measured once, recognised and tuned the next day.

17. Phone Cameras Integration

Phase E: Better cameras and more games

Purpose	Brings the Xiaomi phones into the Camera Manager as front and side cameras.
Inputs	Phone video streams.
Outputs	Multi-camera landmark streams.
Depends on	Camera Manager, Pose Tracker, Action Detector.

Responsibilities

- Register each phone as a camera source with a role.
- Use the front phone as the primary tracker; keep the laptop webcam for setup and menus.
- Use the side phone to confirm jump height and landing.
- Fuse cameras: combine evidence from front and side and fall back to one if the other fails.
- Compare latency and accuracy between cameras and record the findings.

Why it is at this position

It is a quality upgrade, not a prerequisite. Everything works with one camera first, so adding phones later cannot break the base system.

Main risks

- Webcam app limitations with two phones.
- Network or USB latency and bandwidth.
- Fusion rules adding complexity.

Done when

MILESTONE M4: the front phone at 9 to 10 ft is steadier than the laptop camera, and the system survives a camera dropping out.

18. Games 2 and 3

Phase E: Better cameras and more games

Purpose	The real test of reusability: two more games built from the same parts.
Inputs	Existing action events.
Outputs	New games.
Depends on	Mechanics Library, Engine, tracking chain.

Responsibilities

- Game 2: lane switch (lean left, centre, right) using the same tracker and mechanics.
- Game 3: freeze dance or balloon pop with hands, or the TV version of floor is lava.
- Measure how much new code each game needed.
- Feed lessons back into the Mechanics Library.

Why it is at this position

Doing this after the first game and tracking proves the design. If it is painful, the module boundaries need adjusting.

Main risks

- Hidden coupling between Game 1 and the engine.

Done when

MILESTONE M5: each new game takes a fraction of Game 1's effort.

19. Floor Projection Preparation

Phase E: Better cameras and more games

Purpose	Adds a second output target: a projector painting the road and lava on the floor.
Inputs	Game state, calibrations.
Outputs	Projected floor image.
Depends on	Display and UI, Zone and Calibration, a projector purchase.

Responsibilities

- Add a projector display target to Display and UI.
- Calibrate camera to projector so objects appear where the child stands.
- Handle keystone, brightness, shadows and dim-room requirements.
- Keep health and score on the TV (hybrid mode).
- Add a floor-based game mode where obstacles approach in floor space.

Why it is at this position

It needs a mature tracker and a good game first. Spending money before the game is proven would be premature.

Main risks

- Projector brightness in a lit room.
- Camera and projector calibration drift.

Done when

MILESTONE M6: the same game runs on the floor with a one-time calibration.

Part 5: Sequence summary, milestones and review checklist

5.1 Build sequence at a glance

#	Module	Phase	Needs	Result you can see
1	Config and Logging	A	-	Settings file and a log
2	Event Bus	A	1	Test events flow
3	Input Adapter (Keyboard)	A	1, 2	Keys become events
4	Display and UI	B	1, 2	Full-screen TV window and HUD
5	Audio Manager	B	1, 2	Sounds on events
6	Game Engine Core	B	1 to 5	States, loop, difficulty
7	Game Mechanics Library	B	2, 6	Obstacles, health, power-ups
8	Game 1: Runner	B	1 to 7	M1: playable with keyboard
9	Camera Manager	C	1, 2	Steady laptop camera
10	Debug and Replay Tools	C	2, 9	Skeleton overlay, replays
11	Pose Tracker	C	9	Live skeleton
12	Zone and Calibration	C	9, 11	Inside or outside the tiles
13	Player Lock	C	11, 12	One player only
14	Action Detector	C	11 to 13	M2: jump controls the game
15	Profile Manager	D	1, 2, 4	Saved profiles
16	Measurement Capture	D	11, 12, 15	M3: measured, remembered
17	Phone Cameras Integration	E	9, 11, 14	M4: better tracking
18	Games 2 and 3	E	6, 7, 14	M5: reuse proven
19	Floor Projection Prep	E	4, 12	M6: floor-ready

5.2 Why this order, in one paragraph

Foundation first so nothing is hard-coded. Then a full game with the keyboard, so fun and architecture are proven without computer vision. Then the vision chain, in the order a frame travels (camera, debug view, pose, zone, player lock, actions), because each stage needs the one before it. Then identity, which needs a working detector to have anything to store. Finally the upgrades (phones, more games, floor), which are improvements on a system that already works and cannot break it.

5.3 Review checklist: please confirm or change

- Are the 19 modules the right split, or should any be merged or split further?
- Is the keyboard-first order acceptable, or do you want tracking work to start earlier?
- Camera roles: front phone as main tracker, side phone for jump height, laptop for setup and menus. Agree?
- Play zone: 3 tiles wide by 2 deep, centre tile as start. Agree, or prefer a different size?
- Profiles: store numbers and avatar choices only, no face photos, choose name on the TV rather than face recognition. Agree?
- Is one active player at a time acceptable for the first versions (two players later)?
- Sessions and safety rules: should the game enforce a time limit or break reminder?
- Which game should be Game 2 and Game 3?

- Do you want a parent mode (settings, recalibration, profile management) protected from the kids?
- Any missing requirement, module or sequence you want added?

5.4 Next document

After your review we will write the use-case catalogue: for every module, each normal flow, each failure and edge case, the expected outcome, and how the system should handle it. Only after that do we script the modules one at a time, in the order above.

Part 6: Use-case catalogue

Parts 1 to 5 above are unchanged. This part adds every use case for every module, with the success criteria, the failure or edge condition, and how the system should handle it.

6.1 How to read the tables

- **ID:** module number, dot, use case number, for example 14.03 is the third use case of module 14 (Action Detector). Cross-cutting use cases are X.01 and so on.
- **Priority: P1** is required for that module to be considered working and for its milestone. **P2** is required for robust daily use with kids. **P3** is a refinement or later-stage item.
- **Success criteria:** what we observe when the use case works. These become the pass conditions of our tests.
- **Failure or edge condition:** what can go wrong or what unusual situation appears.
- **Handling:** what the system does, and what the family sees. Every handling rule keeps the principle: *fail soft, tell the parent clearly, never freeze the TV.*

6.2 Measurable targets (proposed, tune after first tests)

Measure	Target
Jump detection rate	At least 90 percent of real jumps detected (target for M2); at least 95 percent later
False jump rate	At most 1 per minute during normal play; at most 1 per 5 minutes after tuning
Camera-to-event latency	Under 150 ms with a wired camera; playable up to about 250 ms with forgiving games
Game frame rate	60 FPS steady; never below 45 FPS
Tracking frame rate	20 to 30 FPS; the game stays smooth even if tracking drops
Audio delay (event to sound)	Under about 100 ms perceived; measured with the tap-and-flash test
Player lock accuracy	Wrong-person control in fewer than 1 in 50 sessions
Camera loss recovery	Pause within 1 second; automatic resume within 10 seconds of reconnection
Profile reliability	No profile lost across 100 save and load cycles; corrupt file restored from backup
Startup to playable	Under 60 seconds with a saved profile and calibration
Child self-start	A child can start a game alone without opening a menu in parent mode
Soak test	30 minutes continuous play with flat memory and no crash

6.3 Failure severity and default recovery

Severity	Examples	Default response
Critical	Display lost, camera lost during play, game module error, corrupt profile	Pause or return to menu; clear message on the laptop; automatic recovery attempts; nothing is saved in a half-finished state
Major	Low FPS, tracking confidence low, calibration shifted, audio device missing	Degrade gracefully (lower resolution, skip frames, silent mode), warn the parent, keep playing if safe
Minor	Missing sound file, unknown config key, long name	Use the default, log a warning, no interruption to play
Safety	Child leaves zone, falls, or lies on the floor	Pause immediately or after a short grace period; parent resumes manually

6.4 Module 1: Config and Logging (10 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
01.01 P1	First start with no settings file	Defaults load; a settings file with all defaults is created	Folder is read-only so the file cannot be written	Run on in-memory defaults, log a warning, show a one-line message in the parent view
01.02 P1	Valid settings file loaded	Every module can read its values by name	File is valid but a key is missing	Use the default for that key and log one warning naming the key
01.03 P1	Value out of range (for example volume 300)	Value is rejected or clamped to the allowed range	Silent acceptance would break audio or tracking	Log key, bad value and allowed range; use the default; never crash
01.04 P1	Settings file is malformed or the wrong type	Problem is detected at startup	Half-read file gives random behaviour	Copy the bad file to a .bad backup, load defaults, tell the parent exactly which line failed
01.05 P2	Unknown key in the file (typo)	Typo is spotted	Typo silently ignored so the setting never applies	Log a warning with the closest valid key name
01.06 P2	Command-line override of a setting	Override wins over the file for this run	Override has an invalid value	Reject that override, keep the file value, log it
01.07 P1	Log line written	Line has time, level, module name and message	Disk full or log file locked	Fall back to console only and warn once; the game keeps running
01.08 P2	Log grows large over many sessions	Old logs are rotated and only the last N files are kept	Rotation deletes the newest file by mistake	Rotate by size and age; test with a forced rotation
01.09 P2	Setting changed while the game runs	Change applies only through an explicit reload key	A threshold changes mid-run and the kids' game feels broken	Apply changes at the next game start, except safe items such as volume
01.10 P3	Privacy check of logs	Logs hold events and numbers only	Names, faces or images found in logs	Logs may contain a profile name at most; never images or landmarks

6.5 Module 2: Event Bus (11 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
02.01 P1	Event published with two subscribers	Both receive it in the order published	One subscriber never receives it	Subscriber list checked at publish time; test asserts delivery count
02.02 P1	Event published with no subscribers	Event is dropped quietly	Dropped event hides a wiring mistake	Debug log shows 'no listener' for events that should have one
02.03 P1	A subscriber raises an error	Error is caught and logged; other subscribers still get the event	Whole game crashes	Wrap every delivery; after repeated failures disable that subscriber and warn
02.04 P1	A subscriber is slow	Publisher is not blocked and game stays at 60 FPS	Queue grows without limit	Bounded queue per subscriber; drop oldest for high-rate events, never drop critical events such as GAME_OVER
02.05 P1	Unknown event name published	Rejected with a clear error	Typo creates a ghost event nobody listens to	Only names in the event catalogue are accepted
02.06 P2	Payload missing a required field	Event is rejected or filled with a safe default	Listener crashes on a missing value	Validate payload shape at publish; log source module
02.07 P1	Event published from the camera thread	Delivered safely on the main game thread	Two threads change state at the same time	All cross-thread events pass through a thread-safe queue drained once per frame
02.08 P2	Subscriber unsubscribes during delivery	Delivery finishes cleanly	List changed while looping	Iterate over a copy of the subscriber list
02.09 P2	Tracking floods events (30 per second)	Rates stay stable with no lag	Memory growth or UI lag	Debug tap samples rather than stores every event; high-rate events are coalesced
02.10 P2	Events from two sources arrive out of order	Order is recoverable from timestamps	Game reacts to an old event as if new	Every event carries a source and timestamp; stale events older than a set age are discarded by listeners that care

ID	Scenario	Success criteria	Failure or edge condition	Handling
02.11 P3	Shutdown	Queue is flushed and no new events accepted	Exit hangs on a pending event	Time-limited flush then force stop

6.6 Module 3: Input Adapter (Keyboard) (9 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
03.01 P1	Space pressed	One JUMP event	Key repeat from the OS sends many JUMPs	Ignore auto-repeat; one event per physical press
03.02 P1	Down arrow held then released	DUCK on press, STAND on release	STAND never sent so the character stays ducked	Send STAND on release and on window focus loss
03.03 P1	Left or right arrow pressed and released	LEAN_LEFT or LEAN_RIGHT then LEAN_CENTER	Both keys pressed together	Left and right cancel to LEAN_CENTER
03.04 P1	Jump and lean pressed together	Both events fire	Second key swallowed	Track each key independently
03.05 P1	Window loses focus while a key is held	All held keys are treated as released	Stuck DUCK or lean	Release-all on focus loss; log it
03.06 P2	Key map changed in config	New keys apply at next start	Invalid key name in the file	Fall back to the default key and log a warning
03.07 P2	Parent control keys (pause, quit, debug, recalibrate)	Work at any time	Kids hit them by accident	Parent keys are a separate set that needs a modifier key, for example Ctrl plus key
03.08 P2	Keyboard and camera both active	Both can send events; an on-screen marker shows which is live	Double jump from both sources	Configurable mode: keyboard only, camera only, or both; in real play default to camera only
03.09 P3	Latency simulation switch for testing	Adds a set delay (for example 150 ms) to keyboard events	Games feel fine on keyboard but fail with cameras	Test game difficulty with simulated latency before connecting cameras

6.7 Module 4: Display and UI (13 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
04.01 P1	TV connected as second display at startup	Full-screen window opens on the TV at 1080p	Game opens on the laptop screen instead	Pick the display by config index or by name; log which display was chosen
04.02 P1	TV not detected	Window opens on the laptop screen with a visible warning	Game runs invisibly on a screen nobody watches	Show a banner 'TV not found' on the laptop; keep the game paused at the menu until acknowledged
04.03 P1	HDMI cable pulled or TV turned off mid-game	Game auto-pauses	Game continues with nobody seeing it	Display-lost event pauses; resumes after the display returns plus a 3 second countdown
04.04 P1	4K TV with 1080p render	Image is sharp and 60 FPS	Slow frames from rendering at 4K	Render at 1080p and let the TV upscale; resolution is a setting
04.05 P1	HUD update after an event	Score, health and power-up icons change within one frame	HUD shows old values	HUD reads state from events only and redraws every frame
04.06 P1	Health low	Health bar turns red and a warning sound and icon appear	Warning too subtle from 10 ft	Large bar, high contrast, plus sound
04.07 P2	Text size at 9 to 10 ft	All text readable	Small HUD text unreadable by kids	Minimum font size rule; test from the standing spot
04.08 P2	Profile picker with body control	Lean moves the highlight; jump or a hold selects	Accidental selection	Require a deliberate hold of about one second plus a confirm cue
04.09 P2	Menu with keyboard (parent)	Works at all times	Body control fails and the kids are stuck at a menu	Keyboard always works in menus

ID	Scenario	Success criteria	Failure or edge condition	Handling
04.10 P2	Frame rate drops under 45 FPS	Effects are reduced automatically; a log entry is made	Stutter ruins jump timing	Turn off particles, then lower background detail, then warn
04.11 P2	Photosensitivity	Hit flashes are short and soft	Rapid strobing	No flash faster than 3 per second and no full-screen white flash
04.12 P3	Very long profile name	Name fits or is shortened with an ellipsis	Overlap breaks the layout	Limit name length at creation and clip on display
04.13 P3	Window loses focus or another app pops up on the TV	Game stays on top or pauses	Notification covers the screen	Pause on focus loss; recommend Do Not Disturb

6.8 Module 5: Audio Manager (10 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
05.01 P1	JUMP event	Jump sound plays within one frame	Sound arrives late	Preload all sounds; measure end-to-end audio delay in a test
05.02 P1	Several sounds at once (hit plus pickup plus music)	All mix without cutting each other	Sounds clip or silence the music	Channel limit with priorities: hit and game over highest, ambience lowest
05.03 P1	Sound file missing	Game continues silently for that sound	Crash at load	Log a warning and use a silent placeholder
05.04 P1	No audio output available (TV muted, HDMI audio not selected)	Game still runs	Kids think sound is broken	Startup self-check plays a test tone and asks 'did you hear it?'; message if not
05.05 P2	Looping run sound	Starts on game start, stops on pause or game over	Loop keeps playing in the menu	State changes control loops
05.06 P2	Volume change or mute by the parent	Applies instantly	Volume too loud for kids	Maximum volume cap in config
05.07 P2	Low-health alert	Alert loops while health is low and stops when restored	Alert never stops	Tie alert to health state, not to a one-shot
05.08 P2	Pause and resume	Music and loops pause and continue	Sound continues in the pause screen	Pause event pauses all channels
05.09 P3	Music ducking under effects	Music lowers briefly under key effects	Effects drowned out by music	Automatic ducking amount in config
05.10 P3	Audio latency measurement	A built-in tap-and-flash test reports delay	Sound feels late but nobody knows why	Report in the debug view; recommend wired speaker if over about 120 ms

6.9 Module 6: Game Engine Core (12 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
06.01 P1	Normal start: menu, profile select, calibration, countdown, playing	Each state follows the right one	Skipping a state	State machine allows only listed transitions
06.02 P1	Illegal transition requested	Rejected and logged	Game stuck in a wrong state	Always allow a safe route back to the menu
06.03 P1	Pause by parent key	Everything freezes including timers and sounds	Obstacles keep moving	Pause stops the game clock, not the display
06.04 P1	Player leaves the zone	Auto-pause after a short grace period	Game over while the child is out of the zone	Resume with a 3 second countdown when the player returns
06.05 P1	Health reaches zero	GAME_OVER, then results screen	Game continues at zero health	Single check point after every health change
06.06 P1	Difficulty ramp	Speed rises smoothly with distance up to a cap	Speed climbs without limit	Cap and curve come from config

ID	Scenario	Success criteria	Failure or edge condition	Handling
06.07 P1	Frame time spike	Game catches up without a visible jump	Spiral where catching up causes more lag	Maximum catch-up steps per frame
06.08 P1	Game module throws an error	Engine shows an error screen and returns to the menu	Window freezes or closes	Catch at the engine boundary; log full details
06.09 P2	Player not found for a long time (walked away)	After a timeout the game returns to the menu	Game sits paused forever	Idle timeout in config
06.10 P2	Quit mid-game	Returns to the menu and asks whether to keep the score	Partial run saved as a record	Only completed runs update high scores unless the parent chooses otherwise
06.11 P2	Play again	New game starts quickly with a fresh state	Old objects or power-ups remain	Full reset of game state, keeping the profile
06.12 P3	Session time limit or break reminder	Friendly break message after a set time	Reminder interrupts mid-run	Show only between runs

6.10 Module 7: Game Mechanics Library (15 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
07.01 P1	Jump clears a low obstacle	Points added, streak increases, OBSTACLE_CLEARED	Clear not counted	Count a clear when the player passes the obstacle without a collision
07.02 P1	Player touches an obstacle	Small health loss, hit sound, brief invulnerability	Health lost many times in a row	Invulnerability window of about 1 second
07.03 P1	Two collisions in one frame	Only one hit counted	Double damage	Collision check respects invulnerability
07.04 P1	Jump too early or too late	Player collides; no point	Child thinks the game cheated	Hitboxes forgiving; show a short 'too early' or 'too late' hint in early levels
07.05 P1	Duck obstacle (high bar)	Ducking passes; standing or jumping collides	Wrong action accepted	Each obstacle declares the action that clears it
07.06 P1	Power-up collected	Effect starts with a visible timer; sound plays	Effect never ends	Expiry event always fires
07.07 P1	Two power-ups active	Different types combine; the same type refreshes its timer	Conflicting effects (slow plus speed)	Documented stacking rules per pair
07.08 P1	Shield absorbs a hit	Hit is cancelled and the shield used	Shield also costs health	Shield consumed first
07.09 P1	Health refill at full health	No overflow; bonus points instead	Health above 100 percent	Clamp and convert to points
07.10 P1	Obstacle spawn spacing	Always possible to clear with the player's reaction time	Impossible sequence	Minimum gap computed from speed and jump air time; test generator for impossible patterns
07.11 P2	Streak breaks on a hit	Streak resets and multiplier drops	Streak survives a hit	Reset on OBSTACLE_HIT
07.12 P2	Player still airborne when the next obstacle arrives	Evaluated by position at that moment	Ground assumption causes a wrong hit	Collision uses real vertical position
07.13 P2	Slow-down power-up while speed ramps	Ramp pauses or scales and resumes after	Speed jumps back suddenly	Apply a multiplier on top of the ramp
07.14 P2	Lane obstacle	Collides only if the player is in that lane	Lean threshold borderline	Hysteresis in lane decision
07.15 P3	Obstacles removed off-screen	Memory stays flat over a long run	Memory growth	Remove when off-screen; test a 30 minute run

6.11 Module 8: Game 1: Runner (9 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
08.01 P1	Complete run from start to game over	Score, streak, speed and health behave as designed	Any HUD value wrong at the end	Scripted test run with expected final numbers
08.02 P1	Kid does nothing	Obstacles hit; health falls; game ends	Game never ends	Health loss guarantees an end
08.03 P1	Spamming jump	Jump is limited by air time	Constant jumping avoids all obstacles	Cannot jump again until landing; later a double-jump power-up
08.04 P1	New personal best	Banner and sound; value saved	Best saved on an unfinished run	Update only at game over
08.05 P2	Difficulty presets (easy, normal, hard)	Preset changes speed, spacing and health loss	Presets feel the same	Playtest each preset with the kids
08.06 P2	Speed cap reached	Speed stays constant; obstacle variety increases	Game becomes impossible	Cap per preset
08.07 P2	Tutorial for first-time players	Short guided practice with no health loss	Kids skip and get confused	First run per profile includes a 30 second guided start
08.08 P2	Results screen	Score, best, obstacles cleared, longest streak	Missing stats	All stats computed from the events log
08.09 P3	Play with keyboard versus camera	Same rules and scoring	Different results	Same events; only the source differs

6.12 Module 9: Camera Manager (14 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
09.01 P1	Open the laptop webcam at the set resolution and FPS	Steady frames at the target rate	Camera opens at a different size	Read back the actual size and FPS, log them
09.02 P1	Camera not found	CAMERA_LOST and a clear message	Game waits forever	Fall back to keyboard mode; message 'no camera, using keyboard'
09.03 P1	Camera unplugged mid-game	Pause and a message	Frozen last frame treated as live	Detect stale timestamps; auto-pause
09.04 P1	Camera plugged back in	Reconnect within seconds and resume	Needs a restart	Retry loop every few seconds; resume with a countdown
09.05 P1	Camera busy in another app	Clear message 'camera in use'	Generic error	Detect and name the likely cause
09.06 P1	Frame stops updating but the camera looks alive	Treated as lost	Game keeps using an old frame	Compare timestamps and frame differences
09.07 P2	Camera index changes after reboot or replug	Cameras are matched by name or ID	Wrong camera used for a role	Match by device name; if ambiguous, show a picker and save the choice
09.08 P2	Low FPS reported	CAMERA_LOW_FPS and automatic resolution reduction	Tracking becomes late	Step down resolution; log the change
09.09 P2	Two cameras running	Each in its own thread	One slow camera stalls both	Separate threads with independent buffers
09.10 P2	USB bandwidth shortage with two cameras	Secondary camera is lowered or dropped, primary kept	Primary degrades	Prefer the primary; reduce the secondary first
09.11 P2	OS camera permission denied	Clear instruction to enable it	Silent failure	Detect the error and show steps
09.12 P3	Orientation and mirroring	Each camera has its own rotate and mirror setting	Left and right swapped	Mirror and rotation per role in config; verified in setup
09.13 P3	Auto exposure or auto focus hunting	Stable image	Brightness swings cause tracking jitter	Lock exposure and focus where the app supports it
09.14 P3	Clean shutdown	All cameras released	Camera stays locked after exit	Release in a shutdown handler

6.13 Module 10: Debug and Replay Tools (9 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
10.01 P1	Toggle debug overlay	Skeleton, zone, events and FPS appear; toggling off removes the cost	Overlay slows the game	Overlay drawn only when on and at low cost
10.02 P1	Record a landmark session	File saved with date, config and camera info	Recording missing context	Metadata header on every recording
10.03 P1	Replay a recording	Same events appear every time	Different results on each replay	Deterministic detector; seed any randomness
10.04 P2	Replay with changed thresholds	Report shows event counts before and after	Cannot compare	Side-by-side summary
10.05 P2	Mark true jumps while recording (ground truth key)	Accuracy computed: detected, missed, false	No way to measure quality	Press a key at each real jump; compute rates
10.06 P2	Disk space low	Recording stops with a warning	Disk fills	Check free space before and during recording
10.07 P2	Corrupt recording file	Skipped with an error message	Replay crashes	Validate header and length first
10.08 P3	Optional video recording	Only after an explicit parent action; local only	Video recorded unnoticed	Visible recording indicator; off by default
10.09 P3	Delete recordings	One action deletes them all	Old recordings pile up	Delete-all command in parent mode

6.14 Module 11: Pose Tracker (12 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
11.01 P1	One person fully visible	Landmarks at 20 to 30 FPS	Frame rate too low	Reduce resolution or model size automatically
11.02 P1	Nobody in view	Empty result; PLAYER_LOST after a short delay	Phantom skeleton on furniture	Confidence threshold; require several consecutive frames before declaring a person
11.03 P1	Feet or head cut off	Those points flagged low confidence	Detector uses invented positions	Downstream modules ignore flagged points; show a 'step back' hint
11.04 P1	Several people visible	All returned with separate IDs	Players swapped	Player Lock decides; tracker only reports
11.05 P1	Library fails to load (version mismatch)	Clear error; game continues in keyboard mode	Crash at start	Startup check of versions; message with the fix
11.06 P2	Low light	Confidence falls and a warning appears	Jitter and wrong events	Suggest more light; lower detection sensitivity in dark
11.07 P2	Fast motion blur during a jump	Landmarks hold their last good value for a very short time	Skeleton vanishes mid-air	Short hold (about 100 to 150 ms) then mark lost
11.08 P2	Small child in a big frame	Works but with a warning	Low accuracy on small figures	Hint to move the camera closer or use the front phone camera
11.09 P2	Poster, TV reflection or mirror shows a person	Ignored	Phantom player	Ignore detections outside the zone
11.10 P2	CPU overloaded	Tracking FPS drops before the game does	Game stutters	Frame skipping, then lower resolution, then a warning
11.11 P3	Mirrored image	Left and right mapped correctly	Lean left moves the character right	Mirror flag applied once in a single place
11.12 P3	Loose clothes or jacket	Works	Body shape wrong	Note in measurement; prefer fitted clothes

6.15 Module 12: Zone and Calibration (12 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
12.01 P1	Guided calibration succeeds	Zone and scale saved for this camera	Poor result accepted	Validate against the known 2 ft tile size; accept only within tolerance
12.02 P1	Tape or tiles not visible in the camera	Message explains the problem	Calibration silently wrong	Fallback: parent taps the four zone corners on the screen
12.03 P1	Player inside the zone	PLAYER_IN_ZONE	Flicker between inside and outside	Hysteresis band at the edge
12.04 P1	Player at the edge	Yellow edge warning on the TV	Sudden pause	Warn first, pause only after leaving fully
12.05 P1	Player outside the zone	Pause after a short grace	Instant pause during a jump landing	Debounce for about 1 second; judge by foot position on the floor
12.06 P2	Camera moved or bumped	System detects a shift and asks for recalibration	Wrong zone used all day	Compare reference points at each start; warn on difference
12.07 P2	Scale inconsistent between tiles	Flag the setup	Heights computed wrongly	Reject if tile measurements disagree by more than about 15 percent
12.08 P2	Player jumps	Zone check uses floor position before take-off	Jump looks like leaving the zone	Ignore vertical motion for zone decisions
12.09 P2	Player leans strongly	Still inside	Lean counted as stepping out	Use foot position, not head position
12.10 P2	Separate camera, separate calibration	Each camera has its own saved zone	Wrong calibration applied	Calibration keyed by camera ID
12.11 P3	Lighting change (evening)	Calibration still works	Tape contrast lost	Use high-contrast tape; recheck on startup
12.12 P3	Quick verify on startup	Short check that the saved calibration still fits	Long setup every time	One-step verify with a pass or recalibrate prompt

6.16 Module 13: Player Lock (11 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
13.01 P1	One person stands in the zone	Locked after about half a second of stable presence	Locks too early on someone passing	Require stable presence for a set time
13.02 P1	Two children in the zone	Lock goes to the one nearest the centre tile; message on screen	Wrong child controls the game	Display 'one player at a time' and let the parent choose
13.03 P1	Parent walks through the frame	Lock is unchanged	Control jumps to the parent	Ignore anyone not in the zone and not the locked body
13.04 P1	Sibling briefly blocks the view	Lock held for a short time	Lock lost and game paused too quickly	Hold the lock for about 2 seconds of occlusion
13.05 P1	Player leaves the zone	Lock released after the grace period and game pauses	Lock stays on an empty spot	Release on timeout
13.06 P1	Player jumps out of frame at the top	Lock held through the brief absence	Lock dropped	Short-term prediction using last position
13.07 P2	A different person steps in after a release	New lock and a prompt to confirm who is playing	Scores credited to the wrong profile	Compare body proportions to the active profile; prompt if they differ
13.08 P2	Two people swap positions	Lock follows body proportions	Lock follows the position	Use proportions and tracking history
13.09 P2	Player sits or lies on the floor	Posture ignored for control	Counted as a long duck	Invalid posture produces no action and triggers a pause
13.10 P3	Child carrying a toy or another child	Treated as a single person where possible	Phantom second person	Merge nearby tracks; warn if confusing
13.11 P3	Adult and child both wanted (future two-player)	Not supported in version one	Silent confusion	Clear message; design the lock to hold several players later

6.17 Module 14: Action Detector (17 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
14.01 P1	Real jump	One JUMP within the target delay, then one LAND	Missed jump	Count and display detection rate in the debug view
14.02 P1	Small hop or bounce	Ignored if under the child's threshold	Hop triggers a JUMP	Threshold relative to the child's height; tunable
14.03 P1	Walking or shifting weight	No JUMP	False jumps	Require a fast, clear rise in body position
14.04 P1	Camera noise spike	No event	One bad frame causes a jump	Smoothing and a minimum duration
14.05 P1	Two quick jumps	Two separate JUMP and LAND pairs	Second jump merged or missed	Require a LAND before the next JUMP; refractory time in config
14.06 P1	Duck	DUCK when the body lowers past the threshold for a minimum time; STAND on rising	Tying shoes or crouching counts as a duck	Accepted; the game forgives; log for review
14.07 P1	Lean left and right	LEAN_LEFT, LEAN_RIGHT, then LEAN_CENTER	Flicker at the boundary	Hysteresis: enter at one threshold, leave at a lower one
14.08 P1	Tracking lost	Detector outputs a neutral state	Stuck DUCK or lean	On PLAYER_LOST emit STAND and LEAN_CENTER; no new events until locked again
14.09 P1	Late detection	Latency measured and shown	Jumps feel late	Warn if latency exceeds the target; suggest USB camera, lower resolution or a more forgiving game
14.10 P2	Jump and lean together	Both events fire	One cancels the other	Independent channels
14.11 P2	Tall adult versus small child	Thresholds scale with the profile	Child cannot trigger a jump	Use profile thresholds; default values when none
14.12 P2	Child slouches or posture drifts	Baseline adjusts during calm moments	Baseline drift causes false ducks	Recompute baseline when the player is still
14.13 P2	Arms raised	HANDS_UP and no JUMP	Raising arms looks like a jump	Use hips and feet for jumping; arms only for hand actions
14.14 P2	Stuck state (duck held too long)	Reset after a long time	Child stuck ducked	Auto STAND after about 5 seconds and log
14.15 P2	Parent sensitivity setting (easy, normal, strict)	Changes thresholds in steps	Too sensitive or too hard	Three presets plus a manual value
14.16 P3	Two cameras disagree	Rules decide: front for lean and duck, side for jump height	Conflicting events	Resolve by confidence and role; log disagreements
14.17 P3	Child jumps sideways	JUMP still detected; lean ignored if small	Counted as a lane change	Lean needs a sustained offset, not a transient

6.18 Module 15: Profile Manager (13 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
15.01 P1	Create a profile	Name and avatar saved; profile selectable	Empty or invalid name	Trim spaces; allow letters, numbers and spaces; length limit
15.02 P1	Name already exists	Ask for a different name or load the existing one	Duplicate overwrites	Check on creation; never overwrite silently
15.03 P1	Select a profile	PROFILE_SELECTED; game and detector use its values	Wrong profile used	Show the active name on the HUD
15.04 P1	Profile file corrupt	Restore from the latest backup	Profile lost	Keep a rolling backup; if none, offer to recreate and keep other profiles untouched
15.05 P1	Save after a game	High score updated for that game only	Another game's score overwritten	Per-game score entries

ID	Scenario	Success criteria	Failure or edge condition	Handling
15.06 P2	Rename or change avatar	Changes saved	Scores lost on rename	Profile has a permanent internal ID
15.07 P2	Delete a profile	Parent confirmation then removal including scores	Child deletes a sibling's profile	Delete only in parent mode with confirmation
15.08 P2	Re-measure an existing profile	New measurements saved; old ones kept as backup	Scores reset	Measurements update; scores unchanged
15.09 P2	Guest play	Temporary profile with default values; nothing saved	Scores saved to nobody	Guest mode clearly labelled
15.10 P2	Disk write fails	Profile kept in memory and a warning shown	Changes lost silently	Retry; warn the parent to check storage
15.11 P3	Profile format updated in a later version	Old files are migrated	Old profiles unreadable	Version number in each file plus migration steps
15.12 P3	Many profiles	List stays usable	Long list overflows the screen	Paged or sorted by last played
15.13 P3	Privacy check	No images or face data in files	Face data stored	Numbers and names only; export and delete available

6.19 Module 16: Measurement Capture (12 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
16.01 P1	Guided measurement succeeds	Height, widths, proportions and baseline measured and confirmed	Results unconfirmed	Show results and ask for a confirm before saving
16.02 P1	Player not fully visible	Instruction to step back or forward	Measurement with cut-off limbs	Refuse to measure until the body is in frame
16.03 P1	Child moves during measurement	Measurement restarts with a simple prompt	Garbage numbers saved	Require several stable seconds; use the median
16.04 P1	Child cannot hold the pose after 3 tries	Parent can type height and proportions manually	Child frustrated and quits	Fallback manual entry or default values
16.05 P1	Measurement interrupted (child walks off)	Cancelled and nothing saved	Partial data saved	Atomic save only on completion
16.06 P2	Implausible result (for example 3 metres tall)	Rejected and retried	Bad data accepted	Plausible ranges by age group
16.07 P2	Wrong pose (arms down, sideways)	Prompt with a simple picture	Silent wrong result	Pose check before capture
16.08 P2	Test jumps	Three jumps; median used for the baseline	Fewer than two valid jumps	Fall back to default and offer a retry
16.09 P2	Real height typed by the parent	Used to improve scale	Mismatch with camera estimate	Flag a large mismatch and recheck calibration
16.10 P3	Face helper cannot see the face	Measurement continues without it	Measurement blocked	Face is optional; nothing stored
16.11 P3	Child grows	Reminder to re-measure after a set time	Old measurements cause wrong thresholds	Optional reminder every few months
16.12 P3	Poor lighting	Warning and suggestion	Poor accuracy	Check confidence before and during measurement

6.20 Module 17: Phone Cameras Integration (12 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
17.01 P1	Front phone connected as the main camera	Stream steady; roles assigned	Wrong phone as front	Role by device name with a one-time picker

ID	Scenario	Success criteria	Failure or edge condition	Handling
17.02 P1	Webcam app closed or phone screen locked	Detect lost stream; message 'open the camera app'	Game hangs	Treat as camera loss; fall back to the laptop webcam
17.03 P1	Phone call or notification interrupts	Stream pauses and recovers	Game keeps using stale frames	Stale detection plus auto-pause
17.04 P1	Side phone fails	Game continues with the front camera only	Game stops	Side camera is optional; jump height from the front only
17.05 P1	Front phone fails	Fallback to the laptop webcam with a warning	No tracking	Fallback chain: front phone, laptop webcam, keyboard
17.06 P2	Battery low or phone hot	Warning to the parent	Stream stops mid-game	Keep phones plugged in via USB; show a heat warning
17.07 P2	Latency check	A measured value is shown	Hidden latency breaks timing	Tap-and-flash test; warn if over the target
17.08 P2	Cameras bumped	Recalibration prompt	Wrong geometry	Reference check on startup
17.09 P2	Phone auto-rotates	Orientation locked	Image flips mid-game	Lock orientation in the app and set rotation in config
17.10 P3	Different resolution and FPS per camera	Frames normalised	Mismatch breaks fusion	Resample and timestamp every stream
17.11 P3	Cameras not time-synchronised	Offset estimated and applied	Events from two cameras misaligned	Estimate offset with a shared jump during setup
17.12 P3	Wi-Fi versus USB	USB preferred	Wi-Fi lag	Config option and a warning when on Wi-Fi

6.21 Module 18: Games 2 and 3 (9 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
18.01 P1	Lane game: lean left or right	Character changes lane	Moves on a tiny lean	Lean threshold with hysteresis
18.02 P1	Lane game: leaning at the edge lane	Stays in the lane	Character leaves the road	Clamp at the edges
18.03 P1	Freeze dance: music stops, player freezes	Success if movement stays under the limit	Breathing or small sway counted as moving	Threshold above natural sway; configurable
18.04 P1	Freeze dance: player moves during a freeze	Point lost with a clear message	Unfair to small children	Age-based tolerance in the profile
18.05 P2	Balloon pop: hand touches a balloon	Balloon pops with a sound and points	Hand not visible	Only count visible hands; ignore low-confidence hands
18.06 P2	Hands cross or overlap	Both hands tracked	Left and right swapped	Track by side with a swap check
18.07 P2	Game selection menu	Each game can be chosen and has a short tutorial	Game leaves leftover state	Full teardown on exit
18.08 P2	Per-game high scores	Stored separately in the profile	Scores mix	Key scores by game ID
18.09 P3	How much new code a game needed	Recorded and reviewed	Engine changed for one game	Change the library instead and re-test the earlier games

6.22 Module 19: Floor Projection Preparation (11 use cases)

ID	Scenario	Success criteria	Failure or edge condition	Handling
19.01 P1	Projector detected as another display	Selectable as a floor output	Wrong display chosen	Pick by index or name; show a test pattern

ID	Scenario	Success criteria	Failure or edge condition	Handling
19.02 P1	Camera to projector calibration with a test pattern	Floor objects land where intended within a small tolerance	Objects offset from the player	Validate with markers; accept only under the tolerance
19.03 P1	Room too bright for the pattern	Message asks to dim lights	Calibration fails silently	Brightness check before starting
19.04 P1	Projector moved or bumped	Recalibration prompt	Wrong placement	Reference check on startup
19.05 P2	Shadow of the player on the floor	Important content kept away from under the player	Obstacle hidden under feet	Design rule and visual cue
19.06 P2	Glossy tiles wash out the image	Warning and brightness advice	Poor visibility	Contrast settings; dim room
19.07 P2	One display lost (TV or projector)	Game pauses	Half the game disappears	Pause on any output loss
19.08 P2	Hybrid mode (floor road plus TV HUD)	Both stay in sync	Out of sync	Single game clock for both
19.09 P2	Eye safety	Nobody looks into the lens; projector aimed down	Light in the eyes	Mounting guidance; placement outside the play zone
19.10 P3	Tape zone under projection	Still visible	Tape hidden	Draw the zone on the projection as a guide
19.11 P3	Back to TV-only mode	Same game works	Floor mode becomes mandatory	Output target is a setting

6.23 Cross-cutting and end-to-end use cases (14 use cases)

These use cases involve several modules at once and are the real-world situations a family will meet.

ID	Scenario	Success criteria	Failure or edge condition	Handling
X.01 P1	Normal session: a child walks up, picks their name, stands on the centre tile, plays and sees results	Start to finish with no parent action	A step requires the parent	Time the flow; each step has an obvious prompt
X.02 P1	New child, first time	Create profile, measure, short tutorial, first game	Child abandons at the measurement	Make it a short game-like routine with a manual fallback
X.03 P1	Startup self-check	Config, display, audio, each camera, tracker, profile store, calibration each show pass, warn or fail	Silent failure found mid-game	One screen on the laptop listing all checks with fixes
X.04 P1	Switch between two children	Quick profile change and a new lock	Scores go to the wrong child	Profile switch clears the lock and confirms the active name
X.05 P1	Safe default when anything critical fails	Game falls back to keyboard mode or a clear menu message	Frozen screen	Fail soft for every module
X.06 P2	Sibling runs in front or interferes	Game pauses or ignores the extra person	Wrong child controls	Player Lock plus a visible indicator
X.07 P2	Child slips or falls	Tracking shows an abnormal posture; game pauses	Game continues with a child on the floor	Pause when posture is invalid for a few seconds; parent resumes
X.08 P2	Pet or toy in the zone	Ignored or flagged	Phantom player	Ignore objects that do not match a human pose
X.09 P2	Laptop overheating or performance drop	Tracking FPS lowered first; game stays smooth	Stutter or shutdown	Degradation order: skip frames, lower resolution, reduce effects, warn
X.10 P2	Laptop sleep or notification popups	Game unaffected	Popup covers the TV	Recommend Do Not Disturb and disable sleep while playing
X.11 P2	Evening light changes	Tracking still works	Accuracy falls	Lighting hint and a re-check
X.12 P3	Child who cannot or should not jump	Alternative actions (step, hand raise, lean) mapped to jump	Child excluded	Accessibility profile setting for alternative inputs
X.13 P3	Parent mode	Settings, recalibration, profile management, debug view available behind a parent key combination	Kids change settings	Modifier-key combination; optional passcode
X.14 P3	Long session	Break reminder between runs	Fatigue and injury risk	Time limit setting with friendly prompts

6.24 Counts and coverage

Module	Use cases	P1	P2	P3
1. Config and Logging	10	5	4	1
2. Event Bus	11	6	4	1
3. Input Adapter (Keyboard)	9	5	3	1
4. Display and UI	13	6	5	2
5. Audio Manager	10	4	4	2
6. Game Engine Core	12	8	3	1
7. Game Mechanics Library	15	10	4	1
8. Game 1: Runner	9	4	4	1
9. Camera Manager	14	6	5	3

Module	Use cases	P1	P2	P3
10. Debug and Replay Tools	9	3	4	2
11. Pose Tracker	12	5	5	2
12. Zone and Calibration	12	5	5	2
13. Player Lock	11	6	3	2
14. Action Detector	17	9	6	2
15. Profile Manager	13	5	5	3
16. Measurement Capture	12	5	4	3
17. Phone Cameras Integration	12	5	4	3
18. Games 2 and 3	9	4	4	1
19. Floor Projection Preparation	11	4	5	2
Cross-cutting	14	5	6	3
Total	**235**	**110**	**87**	**38**

6.25 Review questions before scripting

- Are any use cases missing from your point of view (anything your kids do that is not covered)?
- Are the priorities right? Anything marked P2 or P3 that you want as P1?
- Are the measurable targets realistic for your kids and room? We will tune them after the first recordings.
- Should a parent passcode protect settings and profile deletion?
- Should we enforce a session time limit, or only remind?
- After approval, we begin scripting module 1 (Config and Logging) and test it against its use cases 01.01 to 01.10 before moving on.